



Звіт про оригінальність

● Оцінка схожості

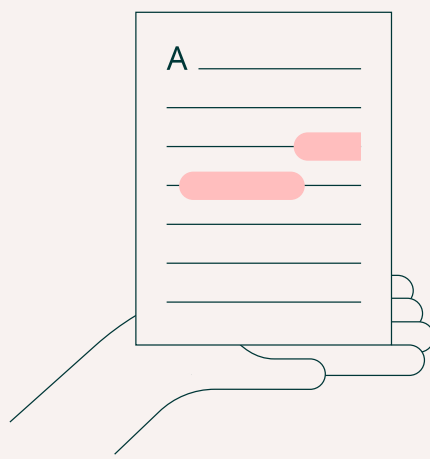
% 5

● Ризик плагіату

СЕРЕДНІЙ

👤 Offiks 🕒 2026-06-11 06:56

Посилання на звіт: 1aVVv / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

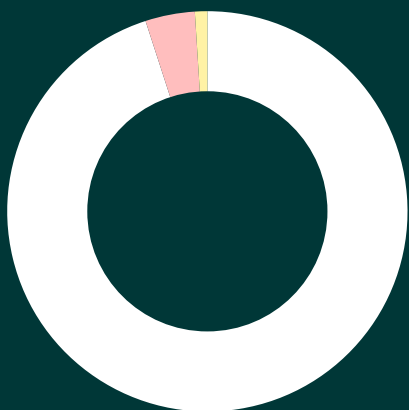
Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



Збіги тексту	4%
Перефразування	1%
Цитований текст	0%
Неправильне цитування	0%
Збігів не знайдено	95%

Ризик плагіату

СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.



5

Збіги

24 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

13 ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. 3 І. МЕЧНИКОВА

3 Факультет 9 математики, фізики та інформаційних технологій Кафедра
математичного забезпечення комп'ютерних систем

9 КУРСОВИЙ ПРОЄКТ

з освітнього компоненту «Програмування»

на тему: «СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ПОРТ»

Студента 1 курсу, 21 денна форма навчання

21 спеціальність F7 «Комп'ютерна інженерія»

Лоева Олександра Олександровича

Керівник: к.т.н., доц. Шестопапов С.В.

22 Національна шкала: _____

22 Кількість балів: _____

ECTS: _____

Дата захисту _____

Члени комісії: _____ Антоненко.О.С

3 (підпис) (прізвище та ініціали)

3 _____ 3 Шестопапов С.В.

3 (підпис) (прізвище та ініціали)

ЗМІСТ

стор.

ВСТУП5

ЗАВДАННЯ6

1.ТЕОРЕТИЧНА ЧАСТИНА7

1.1Базові концепції ООП7

1.2Аналіз об'єктної частини8

2.ПРАКТИЧНА ЧАСТИНА9

2.1 Опис класів9

2.2 Інтерфейс15

ВИСНОВКИ22

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ23

ДОДАТКИ24

ВСТУП

У сучасному світі дуже важливим є автоматизації обліку та зберігання та інформації щодо сучасного флоту, оскільки кількість кораблів дуже велика та одночасно виконуються мільйони записів, змін та стирань різноманітної інформації щодо різних типів кораблів.

Управління сучасним морським портом вимагає високого рівня автоматизації обліку діючих портів. Кожне судно, має базові технічні параметри: унікальний ідентифікаційний номер (індекс), назву, порт приписки, водотоннажність, потужність двигуна та список персоналу тощо.

Мета проєкту – створити програмну реалізацію системи обліку, контролю та зберігання морських суден в порту за допомогою ієрархії класів. Застосунок був зроблений на базі

платформи .NET із використанням мови C#.

ЗАВДАННЯ

Варіант 9а. Створення ієрархії класів на тему «Порт»

Створити ієрархію класів: корабель – базовий клас і пасажирський корабель – похідний. Корабель має потужність двигуна, водотоннажність, назву, порт приписки, екіпаж.

Реалізувати клас член екіпажу з полями ПІБ, посада, вік і стаж та методом змінити посаду. Пасажирський корабель має додаткові поля: кількість пасажирів, кількість шлюпок, місткість шлюпки. Реалізувати метод перевірки, що шлюпок вистачає на пасажирів і членів екіпажу, та метод збільшення кількості шлюпок до мінімально необхідної, якщо їх не вистачає. Реалізувати метод перевірки, що на кораблі є капітан. Також реалізувати клас вантажний корабель з додатковим параметром – вантажопідйомність.

У головному проєкті реалізувати створення, модифікацію та видалення кораблів, а також додавання шлюпок на пасажирські кораблі.

ТЕОРЕТИЧНА ЧАСТИНА

Базові концепції ООП

Об'єктно-орієнтоване програмування (ООП) – це провідна парадигма розробки програмного забезпечення, де основними елементами структури є об'єкти, що взаємодіють між собою. Вона базується на чотирьох фундаментальних принципах, які повноцінно інтегровані в екосистему .NET і мову C#:

абстракція – виділення головних, найбільш значущих характеристик об'єкта та ігнорування другорядних деталей. У роботі абстракція реалізована через створення класу Ship, що містить універсальні параметри судна (наприклад, потужність силової установки, водотоннажність) без прив'язки до його конкретного комерційного чи пасажирського призначення.

інкапсуляція – приховування внутрішньої реалізації об'єкта та захист його полів від прямого несанкціонованого доступу чи псування ззовні. Досягається використанням модифікаторів доступу (таких як private або protected) для полів класу та організацією безпечного доступу через публічні властивості (public Properties із блоками get і set) із вбудованими перевітками умов (наприклад, валідацією вхідного значення value).

успадкування – механізм, що дозволяє створювати нові класи на основі вже існуючих. Це забезпечує повторне використання коду та формування логічних ієрархій типу «є одним із». Спеціалізовані класи CargoShip та PassengerShip успадковують базову логіку

та поля від абстрактного класу Ship, розширюючи її власними специфічними атрибутами.

поліморфізм – Здатність об'єктів **20** з однаковим інтерфейсом поводитися по-різному залежно від конкретного типу об'єкта під час виконання програми. У проєкті реалізовано динамічний поліморфізм через перевизначення (override) абстрактних або віртуальних методів базового класу у класах-нащадках.

Аналіз об'єктної частини

Управління сучасним морським портом вимагає високого рівня автоматизації обліку діючих портів. Кожне судно, має базові технічні параметри: унікальний ідентифікаційний номер (індекс), назву, порт приписки, водотоннажність, потужність двигуна та список персоналу тощо.

Існують різні типи кораблів, наприклад пасажирський та вантажний, що мають свої спільні та індивідуальні характеристики:

пасажирський корабель має кількість пасажирів, сидінь та місткість кожного сидіння. В ньому можна змінювати кількість сидінь на одне, коли місць для пасажирів не вистачає

вантажний корабель має вантажомісткість та величину поточного вантажу, а також перевірку чи місткість не перевищує поточн у кількість вантажу

Екіпаж має своїх працівників із характеристиками, що притаманні звичайному робітникові: прізвище ім'я по-батькові, вік (строго до 16 років), стаж роботи та посаду, що можна змінювати.

Програма дозволяє описувати та змінювати характеристики судна та його екіпажу, враховуючи всі спільні та індивідуальні риси обох типів кораблів, за допомогою інструментів програмування та зручного користувацького інтерфейсу.

ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Структура класів наведена у діаграмі (Додаток А) розглянемо структуру ближче:

1) Абстрактний клас «Ship» (див. лістинг Б.1):

а) Захищені поля:

enginePower (double) – потужність двигуна корабля;

name_of_the_ship (string) – назва корабля;

displacement (double) – водотоннажність корабля;

name_of_the_port (string) – назва порту приписки;

shipIndex (int) – індивідуальний індекс корабля;

crew (List<CrewMember>) – список членів екіпажу

корабля;

static ship (List<Ship>) – статичний список усіх кораблів у системі порту.

b) Конструктори:

Конструктор без параметрів (ініціалізує значеннями за замовчуванням);

Конструктор з параметрами (ініціалізує базові властивості корабля).

c) Властивості (Properties):

EnginePower, NameOfTheShip, Displacement, NameOfThePort – перевіряє на від'ємні значення, порожні рядки та пробіли;

ShipIndex – для читання та запису індексу;

Crew (виключно для читання) – повертає список екіпажу;

Ships (виключно для читання) – повертає загальний список кораблів.

d) Методи:

Абстрактний метод AddShips – додає новий корабель на основі текстового рядка характеристик;

Абстрактний метод AddMembers – додає членів екіпажу на основі текстового рядка;

Абстрактний метод IshaCaptain – перевіряє, чи є капітан на Кораблі;

Абстрактний метод ShipModification – змінює певну

характеристику корабля;

RemoveShipByname – шукає кораблі за назвою та повертає масив із кількістю знайдених кораблів та індексом останнього;

RemoveShipByIndex – видаляє корабель із загального списку за його індивідуальним індексом;

Віртуальний ShowInfo – виводить інформацію про корабль.

2) Клас «PassengerShip», нащадок класу «Ship» (див. лістинг Б.2):

Приватні поля:

number_of_passengers (int) – поточна кількість пасажирів;

number_of_sits (int) – кількість місць (сидінь) на кораблі;

capacity_of_sit (int) – місткість одного пасажирського місця/секції.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Властивості (Properties):

Number_of_passengers, Number_of_sits, Capacity_of_sits – валідацією на від'ємні значення (для місткості місця – також на рівність нулю).

Методи:

IsEnoughSits – перевіряє, чи достатньо місць на кораблі для пасажирів та всього екіпажу;

ChangeNumberOfSits – автоматично розраховує та

збільшує кількість місць, якщо поточних сидінь недостатньо для пасажирів та екіпажу;

Перевизначення AddShips – парсить рядок із 7 параметрів,

створює пасажирський корабель, присвоює випадковий ID (0–1000) та додає його до порту;

Перевизначення AddMembers – додає члена екіпажу

допасажирського корабля (очікує 4 параметри через кому);

Перевизначення IsHasCaptain – перевіряє наявність члена екіпажу з професією "captain" або "Captain";

Перевизначення ShipModification – модифікує одну з 6 характеристик корабля (включаючи специфічні пасажирські властивості) залежно від переданого номера.

3) Клас «CargoShip», нащадок класу «Ship» (див. лістинг Б.3):

а) Приватні поля:

loadCapacity (double) – максимальна вантажопідйомність корабля;

currentLoad (double) – поточна маса завантаженого вантажу;

shipIndex (int) – локальний індекс (дублює або перевизначає логіку ідентифікатора).

б) Конструктори:

Конструктор без параметрів;

Конструктор з параметрами (викликає базовий конструктор base).

в) Властивості (Properties):

LoadCapacity, CurrentLoad – з перевіркою на від'ємні значення вантажу.

г) Методи:

IsOverloading – перевіряє, чи перевищує поточний вантаж максимальну вантажопідйомність корабля;

Перевизначення AddShips – парсить рядок із 6

характеристик, створює вантажний корабель, присвоює випадковий ID та фіксує його в базі порту;

Перевизначення AddMembers – додає члена екіпажу до вантажного корабля (очікує 4 параметри);

Перевизначення IshaCaptain – перевіряє наявність капітана в команді вантажного судна;

Перевизначення ShipModification – модифікує одну з 5 характеристик (включаючи вантажопідйомність та поточну вагу) та автоматично перевіряє корабель на перевантаження;

Перевизначення ShowInfo – виводить інформацію з додатковими властивостями корабля.

е) Властивості:

LoadCapacity, CurrentLoad – з перевіркою на від'ємні значення вантажу;

Перевизначення ShowInfo() – додає власні властивості Класу.

4) Клас «CrewMember» (див. лістинг Б.4):

а) Приватні та захищені поля:

snr (string) – ПІБ (Прізвище, Ім'я, По батькові) члена екіпажу;

profession (protected string) – професія/посада працівника.

age (int) – вік працівника;

term_of_work (int) – стаж (термін) роботи.

b) Конструктори:

Конструктор без параметрів (встановлює значення за замовчуванням, наприклад, "Unknown crew member").

Конструктор з параметрами.

c) Властивості (Properties) та методи:

Proffession – перевіряє, щоб назва професії не була порожньою);

Age – обмежує мінімальний вік працевлаштування (не менше 16 років);

Term_of_work – контролює, щоб стаж не був від'ємним.

SNP – перевіряє формат введення ПІБ (обов'язкова наявність трьох слів: Прізвище, Ім'я, По-батькові);

Change_profession – метод для зміни поточної професії члена екіпажу.

5) Клас «Session» (див. лістинг Б.5):

a) Конструктори:

Конструктор без параметрів.

b) Методи інтерфейсу управління:

ChooseShipYouWantToCange – меню вибору корабля для редагування (за індексом, або автоматично обирає єдиний наявний корабель у порту);

ModifyPassengerShip – консольне підменю дій над пасажирським кораблем (модифікація, зміна сидінь, додавання людей, перевірка капітана, зміна професії).

ModifyCargoShip – інтерфейс для зміни характеристик вантажного корабля;

InterfaceForChangingProfession – інтерактивна зміна

посади конкретного члена екіпажу за його ПІБ (SNP);

InterfaceForChangingProfession – інтерактивна зміна

посади конкретного члена екіпажу за його ПІБ (SNP);

InterfaceForModificationOPfPasi – інтерфейс для зміни характеристик пасажирського корабля;

InterfaceForModificationOPfCargo – покрокове меню

зміни базових та унікальних властивостей вантажного судна;

InterfaceforRemovingSips – Інтерфейс для видалення

кораблів з портової системи (на вибір: за назвою або за унікальним індексом);

InterfaceforRemovingSips – Інтерфейс для видалення

кораблів з портової системи (на вибір: за назвою або за унікальним індексом);

InterfaceForShowingShip – інтерфейс для зображення

характеристик одного корабля або всіх кораблів зі списку.

InterfaceforShowingCrew – інтерфейс для зображення

команди якогось корабля;

InterfaceForRemovingCrew – інтерфейс для видалення члена екіпажа.

6) Клас «Program» (див. лістинг Б.6):

а) Методи:

Main – Організовує нескінченний цикл та виклики всіх методів класу та захопленням помилок роботи текстового інтерфейсу «MAIN PORT MENU».

2.2 Інтерфейс

Готовий проєкт розміщено у додатку Б цього документу. Під час запуску програми користувач бачить вступне вікно головного меню системи керування портами (Рис. 2.1).

Рисунок 2.1 – Головне меню портової системи (MAIN PORT MENU)

Користувач потрапляє сюди одразу після запуску програми. Меню працює в нескінченному циклі, забезпечуючи централізоване керування всіма суднами у порту. На цьому етапі користувачеві доступні 4 основні дії:

додати пасажирський корабель;

додати вантажний корабель;

змінити властивості наявного корабля;

видалити корабель із системи порту.

Якщо користувач обирає пункт «1», програма пропонує ввести характеристику пасажирського судна (Рис.2.2) в один рядок через кому.

Користувач має вказати 7 параметрів: потужність двигуна, назву судна, водотоннажність, порт приписки, поточну кількість пасажирів, кількість сидінь та місткість одного місця.

Рисунок 2.2 – Вікно додавання пасажирського корабля

Після успішного введення система автоматично генерує випадковий індивідуальний індекс судна (ID від 0 до 1000), додає об'єкт до глобального списку та виводить підтвердження: «Your ship with individual index: [ID] 18 was successfully added to the [Назва порту]».

Якщо користувач обирає пункт «2», запускається процес реєстрації

вантажного судна (Рис. 2.3). Користувач вводить 6 характеристик через кому:

потужність двигуна, назву, водотоннажність, порт приписки, максимальну

вантажопідйомність та поточну вагу вантажу.

Рисунок 2.3 – Вікно додавання вантажного корабля

Система валідує дані (перевіряє відсутність від'ємних значень) та

закріплює корабель за портом, присвоюючи йому унікальний індекс.

Якщо користувач обирає пункт «3», керування передається об'єкту класу Session.

Програма аналізує поточний список кораблів у порту: Якщо кораблів немає, виводиться сповіщення: «SYSTEM: 7 You already don't have any ships in port».

Якщо в порту лише один корабель, система автоматично відкриває меню його

модифікації. Якщо кораблів декілька, відкривається вікно вибору (Рис.2.4), де відображаються індекси, назви та типи всіх зареєстрованих суден, а користувачеві пропонується ввести індекс потрібного корабля.

Рисунок 2.4 – Вікно вибору корабля для модифікації

Після вибору пасажирського судна відкривається спеціалізоване підменю дій (Рис. 2.5), яке пропонує 6 опцій:

Рисунок 2.5 – Меню керування пасажирським кораблем

перехід до меню (Рис .2.6) для зміни параметрів пасажирського корабля

Рисунок 2.6 – Меню зміни параметрів пасажирського корабля

автоматичний перерахунок та розширення посадкових місць, якщо пасажирів та членів екіпажу більше, ніж дозволяє поточна

додавання працівника (Рис. 2.7): вводяться ПІБ, професія, вік та стаж через кому;

Рисунок 2.7 – Меню додавання персоналу до корабля

перевірка наявності капітана в екіпажі;

швидка зміна посади працівника за його ПІБ;

повернення до головного меню порту.

Якщо для модифікації було обрано вантажне судно, відкривається відповідне меню (Рис. 2.8). Воно містить 5 функцій:

коригування параметрів включаючи вантажопідйомність та поточну вагу, при введенні некоректних даних завдяки властивостям викликається помилка;

миттєва перевірка судна на перевантаження, якщо вага вантажу перевищує ліміт, програма сповістить про небезпеку

розширення штату Команди вантажного судна;

сканування списку екіпажу на наявність капітана;

змінювання професії члена команди в цьому кораблі;

вихід у головне меню порту.

Рисунок 2.8 – Меню керування вантажним кораблем

Якщо в головному меню обрано пункт «4», користувач існує два випадки, або він потрапляє у майстер видалення суден (Рис. 2.9), або програма викликає помилку, якщо суден немає.

Рисунок 2.9 – Вікно видалення кораблів з порту

Програма у «SHIP REMOVAL WIZARD» пропонує два варіанти очищення бази даних:

видалення за назвою корабля. Програма здійснює пошук у списку, якщо така назва лише одна то програма видаляє корабель з такою назвою із списку, а якщо дві назви збігаються то викликається метод видалення корабля за індексом;

точкове видалення за унікальним ID. Якщо корабель із таким індексом існує, він успішно видаляється із системи, а користувач бачить повідомлення: «You have successfully removed a ship from the port». Якщо індекс введено помилково – виводиться попередження «There are no such ship in the port system».

При виборі пункту «5» викликається меню (Рис. 2.10) у якому пропонується чи буде виведена інформація про всі кораблі зі списку або якогось певного(якщо вони є). При вводі числа «2» виводиться список існуючих кораблів у списку та пропонується ввести індекс потрібного корабля (Рис. 2.11).

Рисунок 2.10 – Меню для вибору способів зображення інформації про кораблі

Рисунок 2.11 – Список існуючих кораблів у списку

При виборі пункту «6» зображується список кораблів у системі портів(якщо вони є) і пропонується ввести індекс бажаного корабля. Після коректного вводу індексу виводиться повна інформація кожного члену екіпажу.

При виборі «7» зображується перелік існуючих кораблів у системі портів, пропонується обрати індекс бажаного корабля, потім зображується перелік членів екіпажу цього корабля та при вводі коректного імені, прізвища та по-батькові видаляється член екіпажу.

При виборі «8» відбувається вихід з програми.

При виникненні будь-яких критичних виключних ситуацій під час роботи з інтерфейсом, глобальний блок try-catch у класі Program перехоплює помилку та виводить на екран безпечно для стабільності утиліти повідомлення: «[CRITICAL ERROR]: [текст помилки] (рис. 2.12)».

Рисунок 2.12 – Помилка

ВИСНОВКИ

У наслідок роботи було успішно виконано поставлене завдання про створення ієрархії класів на тему порт з використанням основних принципів ООП:

використання абстракції та успадкування (через базовий клас Ship

та класи-нащадки CargoShip, PassengerShip) дозволило побудувати гнучку архітектуру програми та уникнути дублювання коду;

принцип інкапсуляції забезпечив надійний захист даних від критичних помилок. Завдяки вбудованим перевіркам у властивостях класів Ship та його нащадків та CrewMember, система унеможливує введення некоректного віку команди (менше 16 років), від'ємної потужності двигунів, порожніх назв портів чи неправильного формату ПІБ;

завдяки поліморфізму реалізовано єдиний інтерфейс модифікації об'єктів за допомогою методу ShipModification();

проведена виправлення логічних помилок (у сеттерах пасажирів, стажу роботи та алгоритмі вилучення елементів) забезпечили високу стабільність і точність роботи застосунку. Програма успішно протидіє некоректному вводу інформації.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

Шилдт Г. 16 С# 4.0: полное руководство. – 2 Пер. с англ. – М. : 26 ООО «И.Д. Вильямс», 2011. 16 – 1056 с.

2 Троелсен Э., Джепикс Ф. Язык программирования С# 7 и платформы .NET и .NET Core. – 8-е изд. – СПб. : Вильямс, 2018. – 1328 с.

Мартин Р. Чистый код: создание, анализ и рефакторинг. – СПб. : Питер, 2019. – 464 с

ДОДАТКИ

ДОДАТОК А - 12 ДІАГРАМА КЛАСІВ

12 Рисунок А.1 – Діаграма класів

12 ДОДАТОК Б 12 - КОД ОСНОВНИХ КЛАСІВ

5 using System;

5 using System.Collections.Generic;

```

5 using System.Linq;
5 using System.Security.Policy;
5 using System.Text;
5 using System.Threading.Tasks;
namespace Project_Programming
{

```

```

28 public abstract class Ship

```

```

{

```

```

protected double enginePower;

```

```

protected string name_of_the_ship;

```

Лістинг Б.1 – абстрактний клас Ship, аркуш 1

```

protected double displacement;

```

```

protected string name_of_the_port;

```

```

private int shipIndex;

```

```

protected List<CrewMember> crew = new List<CrewMember>();

```

```

protected static List<Ship> ship = new List<Ship>();

```

```

protected 15 static Random rand = new Random();

```

```

15 public Ship(double enginePower, string name_of_the_ship, double displacement, string
name_of_the_port)

```

```

{

```

```

EnginePower = enginePower;

```

```

NameOfTheShip = name_of_the_ship;

```

```

NameOfThePort = name_of_the_port;

```

```

Displacement = displacement;

```

```

}

public Ship() : this(0, "Ship", 0, "Port") { }

public double EnginePower

{

get

{

return enginePower;

}

set

{

if (value < 0)

throw new ArgumentException("Power of the engine can't be negative");

enginePower = value;

}

}

public int ShipIndex

{

get { return shipIndex; }

set

{

shipIndex = value;

}

}

public string NameOfTheShip

```

```

{
get
{
return name_of_the_ship;
}
set
{
if (string.IsNullOrEmpty(value) || string.IsNullOrWhiteSpace(value))

Лістинг Б.1 – абстрактний клас Ship, аркуш 2

if (string.IsNullOrEmpty(value) || string.IsNullOrWhiteSpace(value))

throw new ArgumentException("The name of the ship can't be empty or white space");

name_of_the_ship = value;
}
}

public double Displacement
{
get
{
return displacement;
}
set
{
if (value < 0)
{

```

```

throw new ArgumentException("Displacement can't be negative");

}

displacement = value;

}

}

public string NameOfThePort

{

get

{

return name_of_the_port;

}

set

{

if (string.IsNullOrEmpty(value) || string.IsNullOrWhiteSpace(value))

throw new ArgumentException("The name of the port can't be empty or white space");

name_of_the_port = value;

}

}

public List<CrewMember> Crew

{

get { return crew; }

}

public List<Ship> Ships

{

```

```
get { return ship; }
```

```
}
```

```
public abstract Ship AddShips(string character);
```

Лістинг Б.1 – абстрактний клас Ship, аркуш 3

```
public abstract void AddMembers(string str);
```

```
public abstract bool IshaCaptain();
```

```
public abstract void ShipModification(int num, string modification);
```

```
public int[] RemoveShipByname(string name)
```

```
{
```

```
int i = 0;
```

```
int j = 0;
```

```
for (j = 0; j < ship.Count; j++)
```

```
{
```

```
if (ship[j].NameOfTheShip == name)
```

```
{
```

```
i++;
```

```
}
```

```
}
```

```
int[] ints = new int[3];
```

```
ints[0] = i;
```

```
ints[1] = j;
```

```
return ints;
```

```
}
```

```
public bool RemoveShipByIndex(int index)
```

```

{

int i = 0;

for (int j = 0; j < ship.Count; j++)

{

if (ship[j].ShipIndex == index)

{

ship.Remove(ship[j]);

return true;

}

}

return false;

}

public virtual string ShowInfo()

{

return $" name of the ship: {NameOfTheShip}, name of the port: {NameOfThePort}, power of
engine: {EnginePower}, displacement: {Displacement}, Indetificative number: {ShipIndex}";

}

```

Лістинг Б.1 – абстрактний клас Ship, аркуш 4

```

public void ShowCrew()

{

foreach(CrewMember member in crew)

{

member.ToString();

}

}

```

```
}
```

```
}
```

```
}
```

Лістинг Б.1 – абстрактний клас Ship, аркуш 5

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
namespace Project_Programming
```

```
{
```

```
public class PassengerShip : Ship
```

```
{
```

```
private int number_of_passengers;
```

```
private int number_of_sits;
```

```
private int capacity_of_sit;
```

```
public PassengerShip() : this(0, "Unknown", 234, "Unknown", 0, 0, 1) { }
```

```
public PassengerShip(double enginePower, string name_of_the_ship, double displacement,  
string name_of_the_port, int number_of_passengers, int number_of_sits, int capacity_of_sit) :  
base(enginePower, name_of_the_ship, displacement, name_of_the_port)
```

```
{
```

```
Number_of_passengers = number_of_passengers;
```

```
Number_of_sits = number_of_sits;
```

```
Capacity_of_sit = capacity_of_sit;
```

```
}
```



```
public int Number_of_passengers
```

```
{
```

```
get
```

```
{
```

```
return number_of_passengers;
```

```
}
```

```
set
```

```
{
```

Лістинг Б.2 – Клас-Нащадок PassengerShip, аркуш 1

```
if (value < 0)
```

```
{
```

```
throw new ArgumentException("The number of passengers can't be negative");
```

```
}
```

```
number_of_passengers = value;
```

```
}
```

```
}
```

```
public int Number_of_sits
```

```
{
```

```
get
```

```
{
```

```
return number_of_sits;
```

```
}
```

```
set
```

```
{
```

```

if (value < 0)

{

throw new ArgumentException("The number of sits can't be negative");

}

number_of_sits = value;

}

}

public int Capacity_of_sit

{

get

{

return capacity_of_sit;

}

set

{

if (value < 0)

{

throw new ArgumentException("The capacity of sit can't 19 be negative or equal to zero");

}

capacity_of_sit = value;

}

}

public bool IsEnoughSits()

```

```

{

if (Number_of_passengers + crew.Count <= Capacity_of_sit * Number_of_sits)

return true;

return false;

public void ChangeNumberOfSits()

```

Лістинг Б.2 – Клас-Нащадок PassengerShip, аркуш 2

```

{

if (!IsEnoughSits())

{

int a = Number_of_passengers + crew.Count - (Capacity_of_sit * Number_of_sits);

if (a < Capacity_of_sit)

{

Number_of_sits += 1;

}

else

{

Number_of_sits += (a + (Capacity_of_sit - 1)) / Capacity_of_sit;

}

}

}

public override Ship AddShips(string charac)

{

int newIndex;

bool isUnique;

```

```

PassengerShip ShipToAdd = new PassengerShip();

string[] character = charac.Split(',');

if(character.Length == 7)

{

ShipToAdd.NameOfTheShip = character[0];

ShipToAdd.NameOfThePort = character[1];

ShipToAdd.EnginePower = Convert.ToInt32(character[2]);

ShipToAdd.Displacement = Convert.ToInt32(character[3]);

ShipToAdd.Number_of_passengers = Convert.ToInt32(character[4]);

ShipToAdd.Number_of_sits =

Convert.ToInt32(character[5]);

ShipToAdd.Capacity_of_sit = Convert.ToInt32(character[6]);

}

else

{

throw new ArgumentException("You have entered data incorrectly");

}

do

{

newIndex = rand.Next(1, 1000);

Лістинг Б.2 – Клас-Нащадок PassengerShip, аркуш 3

isUnique = true;

foreach (Ship s in ship)

{

```

```

if (s.ShipIndex == newIndex)

{

isUnique = false;

break;

}

}

} while (!isUnique);

ShipToAdd.ShipIndex = newIndex;

ship.Add(ShipToAdd);

return ShipToAdd;

}

8 public override void AddMembers(string str)

{

CrewMember member = new CrewMember();

string[] character = str.Split(',');

if (character.Length == 4)

{

member.SNP = character[0];

member.Age = Convert.ToInt32(character[1]);

member.Term_of_work = Convert.ToInt32(character[2]);

member.Proffession = character[3];

crew.Add(member);

}

else

```

```

{
    throw new ArgumentException("You have written string inccorectly");
}
}

```

```

8 public override bool IshaCaptain()

```

```

{
    int i = 0;
    for (i = 0; i < crew.Count; i++)
    {
        if (crew[i].Proffession == "captain" || crew[i].Proffession == "Captain")

```

Лістинг Б.2 – Клас-Нащадок PassengerShip, аркуш 4

```

{
    i = -1;
    break;
}
}

if (i == -1)
{
    return true;
}

return false;
}

```

```

8 public override void ShipModification(int num, string modification)

```

```

{

```

```
switch (num)
{
case 1:
{
EnginePower = Convert.ToDouble(modification);
break;
}
case 2:
{
Displacement = Convert.ToDouble(modification);
break;
}
case 3:
{
NameOfTheShip = modification;
break;
}
```

Лістинг Б.2 – Клас-Нащадок PassengerShip, аркуш 5

```
case 4:
{
Number_of_passengers = Convert.ToInt32(modification);
break;
}
case 5:
```

```

{
    Number_of_sits = Convert.ToInt32(modification);

    break;
}

case 6:
{
    Capacity_of_sit = Convert.ToInt32(modification);

    break;
}

default:
{
    throw new ArgumentException("You have entered an inccorect number");
}

}

}

}

public override string ShowInfo()
{
    return $"{base.ShowInfo()} number of passengers: {Number_of_passengers}, number of sits:
    {Number_of_sits}, capacity of sit: {Capacity_of_sit}";
}

}

}

```

Лістинг Б.2 – Клас-Нащадок PassengerShip, аркуш 6

```
using System;
```



```

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using static System.Windows.Forms.VisualStyles.VisualStyleElement.Rebar;

namespace Project_Programming

{

    public class CargoShip : Ship

    {

        private double loadCapacity;

        private double currentLoad;

        public CargoShip() : this(0, "Unknown", 0, "Unknown", 0, 0) { }

        public CargoShip(double enginePower, string name_of_the_ship, double displacement, string
name_of_the_port, double loadCapacity, double load) : base(enginePower, name_of_the_ship,
displacement, name_of_the_port)

        {

            LoadCapacity = loadCapacity;

            CurrentLoad = load;

        }

        public double LoadCapacity

        {

            get { return loadCapacity; }

            set

            {

                if (value < 0)

```

```

{
    throw new ArgumentException("Load capacity can't be negative");
}

loadCapacity = value;
}
}

public double CurrentLoad
{
    get { return currentLoad; }
    set
    {
        if (value < 0)
        {
            throw new ArgumentException("Load can't be negative");
        }

        currentLoad = value;

```

Лістинг Б.3 – Клас-нащадок CargoShip, аркуш 1

```

}

}

public bool IsOverloading()
{
    if (CurrentLoad > LoadCapacity)
    {
        return true;

```

```

}

else

{

return false;

}

}

public override Ship AddShips(string charac)

{

CargoShip ShipToAdd = new CargoShip();

string[] character = charac.Split(',');

int newIndex;

bool isUnique;

ShipToAdd.NameOfTheShip = character[0];

ShipToAdd.NameOfThePort = character[1];

ShipToAdd.EnginePower = Convert.ToInt32(character[2]);

ShipToAdd.Displacement = Convert.ToInt32(character[3]);

ShipToAdd.LoadCapacity = Convert.ToInt32(character[4]);

ShipToAdd.CurrentLoad = Convert.ToInt32(character[5]);

do

{

newIndex = rand.Next(1, 1000);

isUnique = true;

foreach (Ship s in ship)

```

```

{
if (s.ShipIndex == newIndex)
{
isUnique = false;
break;
}
}
} while (!isUnique);

```

Лістинг Б.3 – Клас-нащадок CargoShip, аркуш 2

```

ShipToAdd.ShipIndex = newIndex;
ship.Add(ShipToAdd);
return ShipToAdd;
}

public override void AddMembers(string str)
{
CrewMember member = new CrewMember();
string[] character = str.Split(',');
if (character.Length == 4)
{
member.SNP = character[0];
member.Age = Convert.ToInt32(character[1]);
member.Term_of_work = Convert.ToInt32(character[2]);
member.Proffession = character[3];
crew.Add(member);
}
}

```

```

}

else

{

throw new ArgumentException("You have entered data incorrectly");

}

}

30 public override bool IShasCaptain()

{

int i = 0;

for (i = 0; i < crew.Count; i++)

{

if (crew[i].Profession == "captain" || crew[i].Profession == "Captain")

{

i = -1;

break;

}

}

if (i == -1)

{

return true;

}

return false;

}

```

Лістинг Б.3 – Клас-нащадок CargoShip, аркуш 3

```
public override void ShipModification(int num, string mod)
{
    switch (num)
    {
        case 1:
        {
            EnginePower = Convert.ToDouble(mod);
            break;
        }
        case 2:
        {
            Displacement = Convert.ToDouble(mod);
            break;
        }
        case 3:
        {
            NameOfTheShip = mod;
            break;
        }
        case 4:
        {
            LoadCapacity = Convert.ToDouble(mod);
            break;
        }
    }
}
```

```

case 5:

{

CurrentLoad = Convert.ToDouble(mod);

IsOverloading();

break;

}

default:

{

throw new ArgumentException("You have entered data incorrectly");

}

}

}

public override string ShowInfo()

{

```

Лістинг Б.3 – Клас-нащадок CargoShip, аркуш 4

```

return $"{base.ShowInfo()} load capacity: {LoadCapacity} current load: {CurrentLoad}";

}

}

}

```

Лістинг Б.3 – Клас-нащадок CargoShip, аркуш 5

```

1 using System;

1 using System.Collections.Generic;

1 using System.Linq;

1 using System.Runtime.CompilerServices;

```

```

1 using System.Text;

1 using System.Threading.Tasks;

1 namespace Project_Programming
{

public class CrewMember
{

private string snp;

protected string profession;

private int age;

private int term_of_work;

public CrewMember() : this("Unknown crew member", "Unknown", 20, 2) { }

public CrewMember(string Snp, string proffesion, int age0, int Term)

{

SNP = Snp;

Proffession = proffesion;

Age = age0;

Term_of_work = Term;

}

public void Change_profession(string profession)

{

Proffession = profession;

}

public string Proffession

{

```



```
get { return profession; }
```

```
set
```

```
{
```

```
if (string.IsNullOrEmpty(value))
```

Лістинг Б.3 – Клас CrewMember, аркуш 1

```
throw new ArgumentNullException("Proffesion can't be empty");
```

```
profession = value;
```

```
}
```

```
}
```

```
25 public int Age
```

```
25 {
```

```
get { return age; }
```

```
set
```

```
{
```

```
if (value < 16)
```

```
{
```

```
throw new ArgumentException("Age can't be lesser than 16");
```

```
}
```

```
age = value;
```

```
}
```

```
}
```

```
public int Term_of_work
```

```
{
```

```
get { return term_of_work; }
```

```
set
{
if (value < 0)
{
throw new ArgumentException("The term of the work can't be negative or");
}
else if (value >= (Age - 16))
{
throw new ArgumentException("The term of the work can't be greater than age of the man");
}
else
{
term_of_work = value;
}
}
}

public string SNP
{
get { return snp; }
set
{
string[] str = value.Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);

if (str.Length < 3)
```

Лістинг Б.3 – Клас CrewMember, аркуш 2

```

{

throw new ArgumentException("Full name of the peson must be defined as: Surname name
patronymic");

}

else if (string.IsNullOrEmpty(str[0]) || string.IsNullOrEmpty(str[1]) ||
string.IsNullOrEmpty(str[2]))

{

throw new ArgumentException("Full name of the peson must be defined as: Surname name
patronymic");

}

snp = value;

}

}

14 public override string ToString()

14 {

return $"SNP: {SNP}, age: {Age}, term of work {term_of_work}, profession {Proffession}";

}

}

}

```

Лістинг Б.4 – Клас CrewMember, аркуш 3

```

1 using System;

1 using System.Collections.Generic;

1 using System.Linq;

1 using System.Runtime.CompilerServices;

1 using System.Text;

```

```
1 using System.Threading.Tasks;
```

```
1 namespace Project_Programming
```

```
{
```

```
public class Session
```

```
{
```

```
public Session() { }
```

```
public void ChooseShipYouWantToCange()
```

```
{
```

```
Ship template = new PassengerShip();
```

```
if (template.Ships.Count == 0)
```

```
if (template.Ships.Count == 0)
```

```
{
```

```
Console.WriteLine("\n\t+-----+");
```

Лістинг Б.5 – Клас Session, аркуш 1

```
Console.WriteLine("\t| SYSTEM: 7 You already don't have any ships in port. |");
```

```
Console.WriteLine("\t+-----+\n");
```

```
}
```

```
else if (template.Ships.Count == 1)
```

```
{
```

```
if (template.Ships[0] is PassengerShip)
```

```
{
```

```
ModifyPassengerShip(template.Ships[0] as PassengerShip);
```

```
}
```

```
else
```

```

{
    ModifyCargoShip(template.Ships[0] as CargoShip);
}

}

else
{
    foreach (Ship ship1 in template.Ships)
    {
        Console.WriteLine($"{ship1.NameOfTheShip} index: {ship1.ShipIndex}");
    }

    Console.WriteLine("\n+-----+");

    Console.WriteLine("| SHIP SELECTION BOARD |");

    Console.WriteLine("+-----+");

    Console.Write(" Print an individual index of your ship: ");

    int Inindex =

    Convert.ToInt32(Console.ReadLine());

    foreach (Ship ship in template.Ships)
    {
        if (ship.ShipIndex == Inindex)
        {
            if (ship is PassengerShip)
            {
                ModifyPassengerShip(ship as PassengerShip);
            }
        }
    }
}

```

```

}

else

{

ModifyCargoShip(ship as CargoShip);

}

break;

```

Лістинг Б.5 – Клас Session, аркуш 2

```

}

}

}

}

public void ModifyPassengerShip(PassengerShip pass)

{

while (true)

{

Console.WriteLine("\n+-----+");

Console.WriteLine($" | PASSENGER SHIP MANAGEMENT: {pass.NameOfTheShip,-46} |");

Console.WriteLine("+-----+");

Console.WriteLine($" | 1. Modify a property of your ship |");

Console.WriteLine($" | 2. Change number of sits of your ship |");

Console.WriteLine($" | 3. Add members to your ship |");

Console.WriteLine($" | 4. Check if the {pass.NameOfTheShip} has captain |");

Console.WriteLine($" | 5. Change profession of your crew member |");

Console.WriteLine($" | 6. Exit |");

```

```

Console.WriteLine("+-----+");

Console.Write(" Enter a number to do something: ");

string input = Console.ReadLine();

Console.Clear();

if (input == "6")

break;

switch (input)

{

case "1":

Console.WriteLine("\n[Modifying property...]");

InterfaceForModificationOPfPass(pass);

break;

case "2":

Console.WriteLine("\n[Changing number of seats...]");

pass.ChangeNumberOfSits();

break;

Лістинг Б.5 – Клас Session, аркуш 3

case "3":

Console.WriteLine("\n[Adding members...]");

Console.WriteLine(" Print with separating by ',' characteristics of your member:");

Console.WriteLine(" SNP, age, term of work, proffesion");

Console.Write(" -> ");

string str = Console.ReadLine();

pass.AddMembers(str);

```

```

break;

case "4":

Console.WriteLine("\n[Checking for captain...]");

if (pass.IshasCaptain())

{

Console.WriteLine("\n\t=== Your ship have captain ===");

}

else

{

Console.WriteLine("\n\t=== Your ship don't have captain ===");

}

break;

case "5":

{

Console.WriteLine("\n[Changing profession...]");

InterfaceForChangingProfession(pass);

break;

}

default:

Console.WriteLine("\n[!] Invalid input. Please enter a number from 1 to 6.");

break;

}

}

public void ModifyCargoShip(CargoShip pass1)

```



```

{
while (true)
{
Console.WriteLine("\n+-----+");

Лістинг Б.5 – Клас Session, аркуш 4

Console.WriteLine($" | CARGO SHIP MANAGEMENT: {pass1.NameOfTheShip,-50} | ");

Console.WriteLine("+-----+");

Console.WriteLine(" | 1. Modify a property of your ship | ");

Console.WriteLine(" | 2. Check if your ship is overloaded | ");

Console.WriteLine(" | 3. Add members to your ship | ");

Console.WriteLine($" | 4. Check if the {pass1.NameOfTheShip} has captain | ");

Console.WriteLine(" | 5. Change proffesion of your crew member | ");

Console.WriteLine(" | 6. Exit | ");

Console.WriteLine("+-----+");

Console.Write(" Enter a number to do something: ");

string input = Console.ReadLine();

Console.Clear();

if (input == "6")

break;

switch (input)

{

case "1":

Console.WriteLine("\n[Modifying property...]");

InterfaceForModificationOPfCargo(pass1);

```

```

break;

case "2":

Console.WriteLine("\n[Checking...]");

if (pass1.IsOverloading())

{

Console.WriteLine("Your ship is overloaded");

}

else

{

Console.WriteLine("Yoyr ship is not overloaded");

}

break;

case "3":

{

Console.WriteLine("\n[Adding members...]");

Console.WriteLine(" Print with separating by ',' characteristics of your member:");

Console.WriteLine(" SNP, age, term of work, proffesion");

Лістинг Б.5 – Клас Session, аркуш 5

Console.Write(" -> ");

string str = Console.ReadLine();

pass1.AddMembers(str);

break;

}

case "4":

```

```

Console.WriteLine("\n[Checking for captain...]");

if (pass1.IshasCaptain())

{

Console.WriteLine("\n\t=== Your ship have captain ===");

}

else

{

Console.WriteLine("\n\t=== Your ship don`t have captain ===");

}

break;

case "5":

{

Console.WriteLine("\n[Changing

proffession...]");

InterfaceForChangingProfession(pass1);

break;

}

default:

Console.WriteLine("\n[!] Invalid input. Please enter a number from 1 to 6.");

break;

}

}

}

```

```

public bool InterfaceForChangingProfession(Ship ship)
{
    if (ship.Crew.Count == 0)
    {
        Console.WriteLine("\n\t+-----+");
        Console.WriteLine("\t| ----- This ship has not members ----- |");
        Console.WriteLine("\t+-----+\n");
        return false;
    }

```

Лістинг Б.5 – Клас Session, аркуш 6

```

    }

    else
    {
        foreach (var item in ship.Crew)
        {
            Console.WriteLine($"{item.ToString()}");
        }

        Console.WriteLine("\n Print SNP of member 11 which profession you want to change: ");
        CrewMember currentMember = new CrewMember();

        string snp = Console.ReadLine();

        foreach (var a in ship.Crew)
        {
            if (a.SNP == snp)
            {
                currentMember = a;
            }
        }
    }
}

```

```

}

}

if (currentMember.SNP == 29 "Unknown crew member")

29 {

Console.WriteLine("\n\t[!] There are not such crew member");

return false;

}

Console.Write($" Print a profession which will be owned by {currentMember.SNP}: ");

string prof = Console.ReadLine();

currentMember.Change_profession(prof);

Console.WriteLine("\n\t+-----+");

Console.WriteLine("\t| ----- Successfully changed profession ----- |");

Console.WriteLine("\t+-----+\n");

return true;

}

}

public void InterfaceForModificationOPfPass(PassengerShip pass)

{

int k = 0;

while (true)

{

if (k > 0)

{

Console.WriteLine("\n+-----+");

```

```
Console.WriteLine("| Do 11 you want to modify another property? |");
```

Лістинг Б.5 – Клас Session, аркуш 7

```
Console.WriteLine("+-----+");
```

```
Console.WriteLine("| 1. Yes |");
```

```
Console.WriteLine("| 2. No |");
```

```
Console.WriteLine("+-----+");
```

```
Console.Write(" Choose option: ");
```

```
int j = Convert.ToInt32(Console.ReadLine());
```

```
if (j == 2)
```

```
break;
```

```
}
```

```
k++;
```

```
Console.WriteLine("\n+-----+");
```

```
Console.WriteLine("| CHARACTERISTIC OF YOUR PASSENGER SHIP TO MODIFY |");
```

```
Console.WriteLine("+-----+");
```

```
Console.WriteLine("| 1. Engine power (in watts) |");
```

```
Console.WriteLine("| 2. Displacement
```

```
|");
```

```
Console.WriteLine("| 3. Name of the ship |");
```

```
Console.WriteLine("| 4. Number of passengers |");
```

```
Console.WriteLine("| 5. Number of sits |");
```

```
Console.WriteLine("| 6. Capacity of sit |");
```

```
Console.WriteLine("| 7. Quit |");
```

```
Console.WriteLine("+-----+");
```

```

Console.Write(" Enter parameter index: ");

int i = Convert.ToInt32(Console.ReadLine());

Console.Clear();

if (i == 7)

break;

Console.Write(" Print the new parameter of property: ");

string name = Console.ReadLine();

pass.ShipModification(i, name);

Console.WriteLine($"\\n\\t=== You have modified property of {pass.NameOfTheShip} ===");

}

```

Лістинг Б.5 – Клас Session, аркуш 8

```

}

public void InterfaceForModificationOPfCargo(CargoShip pass)

{

int k = 0;

while (true)

{

if (k > 0)

{

Console.WriteLine("\\n+-----+");

Console.WriteLine("| Do you want to modify another property? |");

Console.WriteLine("+-----+");

Console.WriteLine("| 1. Yes |");

Console.WriteLine("| 2. No |");

```

```

Console.WriteLine("+-----+");

Console.Write(" Choose option: ");

int j = Convert.ToInt32(Console.ReadLine());

if (j == 2)

break;

}

k++;

Console.WriteLine("\n+-----+");

Console.WriteLine("| CHARACTERISTIC OF YOUR CARGO SHIP TO MODIFY |");

Console.WriteLine("+-----+");

Console.WriteLine("| 1. Engine power (in watts) |");

Console.WriteLine("| 2. Displacement |");

Console.WriteLine("| 3. Name of the ship |");

Console.WriteLine("| 4. Load Capacity |");

Console.WriteLine("| 5. Current load |");

Console.WriteLine("| 6. Quit |");

Console.WriteLine("+-----+");

```

ЛІстинг Б.5 – Клас Session, аркуш 9

```

Console.Write(" Enter parameter index: ");

int i = Convert.ToInt32(Console.ReadLine());

if (i == 6)

break;

Console.Write(" Print the new parameter of property: ");

```



```

string name = Console.ReadLine();

pass.ShipModification(i, name);

Console.WriteLine($"\\n\\t=== You have modified property of {pass.NameOfTheShip} ===");

}

}

public void InterfaceforRemovingSips()

{

Ship templateShip = new PassengerShip();

if (templateShip.Ships.Count == 0)

{

Console.Clear();

Console.WriteLine("\\n\\t+-----+");

Console.WriteLine("\\t| ----- There are not

ships in the port system ----- |");

Console.WriteLine("\\t+-----+\\n");

}

else

{

Console.WriteLine("\\n+-----+");

Console.WriteLine("| SHIP REMOVAL WIZARD |");

Console.WriteLine("+-----+");

ships in the port system ----- |");

Console.WriteLine("\\t+-----+\\n");

}

```

```

else

{

Console.WriteLine("\n+-----+");

Console.WriteLine(" | SHIP REMOVAL WIZARD |");

Лістинг Б.5 – Клас Session, аркуш 10

Console.WriteLine("+-----+");

Console.WriteLine(" | 1. 23 By the name of the ship |");

Console.WriteLine(" | 2. By the individual index of your ship |");

Console.WriteLine("+-----+");

Console.Write(" Choose mechanism: ");

int k = Convert.ToInt32(Console.ReadLine());

Console.Clear();

switch (k)

{

case 1:

{

Console.WriteLine("There are ships in your port system");

foreach (Ship s in templateShip.Ships)

{

{

Console.WriteLine($"{s.NameOfTheShip} Name: {s.NameOfTheShip}");

}

}

}

Console.Write("\n Print 17 the name of your ship: ");

```

```

string name = Console.ReadLine();

int[] count = templateShip.RemoveShipByname(name);

Console.Clear();

if (count[0] == 1)

{ templateShip.Ships.RemoveAt(count[1] - 1);

Console.WriteLine("\n\t=== Ship removed successfully ===");

}

else if (count[0] == 0)

{

Console.WriteLine("\n\t[!] There are not such ship in the port system");

}

else

{

Console.WriteLine("\n\t[!] There are several ships with the same name");

```

Лістинг Б.5 – Клас Session, аркуш 11

```

Console.WriteLine("There are ships in your port system");

foreach (Ship s in templateShip.Ships)

{

{

Console.WriteLine($"{s.NameOfTheShip} Index: {s.ShipIndex}");

}

}

Console.Write(" Print the individual index of your ship: ");

int index = Convert.ToInt32(Console.ReadLine());

```

```

Console.Clear();

if ((templateShip.RemoveShipByIndex(index)))

{

Console.Clear();

Console.WriteLine("\n\t=== You have successfully removed a ship from the port ===");

}

else

{

Console.WriteLine("\n\t[!] There are not such ship in the port system");

}

}

break;

}

case 2:

{

Console.WriteLine("There are ships in your port system");

foreach (Ship s in templateShip.Ships)

{

{

Console.WriteLine($"{s.NameOfTheShip} Index: {s.ShipIndex}");

}

}

Console.WriteLine("\n Print the individual index of your ship: ");

int index = Convert.ToInt32(Console.ReadLine());

```

```
Console.Clear();
```

Лістинг Б.5 – Клас Session, аркуш 12

```
if ((templateShip.RemoveShipByIndex(index)))
```

```
{
```

```
Console.Clear();
```

```
Console.WriteLine("\n\t=== You have successfully removed a ship from the port ===");
```

```
}
```

```
else
```

```
{
```

```
Console.Clear();
```

```
Console.WriteLine("\n\t[!] There are no such ship in the port system");
```

```
}
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
public void InterfaceOfShowingOfTheShip()
```

```
{
```

```
Ship ship = new PassengerShip();
```

```
if (ship.Ships.Count == 0)
```

```
{
```

```
Console.WriteLine("There are not ships in the port system");
```

```
}
```

```

else if (ship.Ships.Count == 1)

{

Console.WriteLine(ship.Ships[0].ShowInfo());

}

else

{

Console.WriteLine("What do you want to do?");

Console.WriteLine("1.Show information about each of ships");

Console.WriteLine("2.Show information about particular ship");

Console.WriteLine("3.Exit");

Console.WriteLine("Print 14 the number of option ypu want to choose");

string str = Console.ReadLine();

switch (str)

{

case "1":

{

foreach (Ship sh in ship.Ships)

Лістинг Б.5 – Клас Session, аркуш 13

{

{

Console.WriteLine($"{sh.ShowInfo()}");

Console.Write("\n");

}

}

}

```

```

break;

}

case "2":

{

Console.WriteLine("There are ships in your port system");

foreach (Ship s in ship.Ships)

{

{

Console.WriteLine($"{s.NameOfTheShip} Index: {s.ShipIndex}");

}

}

Console.WriteLine("Print the index of ship you want watch information about it:");

int index = Convert.ToInt32(Console.ReadLine());

foreach (Ship sh in ship.Ships)

{

{

if (sh.ShipIndex == index)

{

Console.WriteLine($"{sh.ShowInfo()}");

break;

}

}

}

}

```

```

break;

}

case "3":

{

break;

}

default:

{

Console.WriteLine("You have entered an inccorrecct number");

break;

```

Лістинг Б.5 – Клас Session, аркуш 14

```

}

}

}

}

public void InterfaceForShowingMember()

{

Ship ship = new PassengerShip();

if (ship.Ships.Count == 0)

{

Console.WriteLine("There are not ships in the port system");

}

else

{

```



```

Console.WriteLine("There are ships in your port system");

foreach (Ship s in ship.Ships)
{
    Console.WriteLine($"{s.NameOfTheShip} Index: {s.ShipIndex}");
}

Console.WriteLine("Print the index of ship you want watch information about his members:");

int index = Convert.ToInt32(Console.ReadLine());

foreach (Ship sh in ship.Ships)
{
    if (sh.ShipIndex == index)
    {
        if (sh.Crew.Count == 0)
        {
            Console.WriteLine("There are not members in this crew");
            break;
        }

        foreach (CrewMember m in sh.Crew)
        {
            Console.WriteLine($"{m.ToString()}");
        }

        break;
    }
}

Лістинг Б.5 – Клас Session, аркуш 15
}

```

```

}

}

public void InterfaceForRemovingMembers()

{

    Ship ship = new PassengerShip();

    if (ship.Ships.Count == 0)

    {

        Console.WriteLine("There are not ships in the port system");

    }

    else

    {

        Console.WriteLine("There are ships in your port system");

        foreach (Ship s in ship.Ships)

        {

            Console.WriteLine($"{s.NameOfTheShip} Index: {s.ShipIndex}");

        }

        Console.WriteLine("Print the index of ship you want remove member:");

        int index = Convert.ToInt32(Console.ReadLine());

        foreach (Ship sh in ship.Ships)

        {

            if (sh.ShipIndex == index)

            {

                if (sh.Crew.Count == 0)

                {

```

```

Console.WriteLine("There are not members in this crew");

break;

}

Console.WriteLine("There are members in ship");

foreach (CrewMember m in sh.Crew)

{

Console.WriteLine($"{m.ToString()}");

Console.WriteLine("Enter an snp of member 4 you want to remove");

string i = Console.ReadLine();

}

for (int i2 = 0; i2 < ship.Crew.Count; i2++)

{

Лістинг Б.5 – Клас Session, аркуш 16

if (ship.Crew[i2].SNP == i)

{

ship.Crew.RemoveAt(i2);

break;

}

}

break;

}

}

}

}

```

```
}
```

```
}
```

Лістинг Б.5 – Клас Session, аркуш 17

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Runtime;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
namespace Project_Programming
```

```
{
```

```
27 public class Program
```

```
27 {
```

```
1 static void Main(string[] args)
```

```
1 {
```

```
int i = 0;
```

```
Session session = new Session();
```

```
try
```

```
{
```

```
while (true)
```

```
{
```

```
string k = string.Empty;
```

```
if (i == 0)
```

```
{
```

```
Console.WriteLine("=====
=====");
```

```
Console.WriteLine("Hi, it is your port system. Which 4 action do you want to do?");
```

Лістинг Б.6 – Клас Program, аркуш 1

```
Console.WriteLine("=====
=====");
```

```
}
```

```
i++;
```

```
Console.WriteLine("\n+-----+");
```

```
Console.WriteLine("| MAIN PORT MENU |");
```

```
Console.WriteLine("+-----+");
```

```
Console.WriteLine("| 1. Add Passenger ship |");
```

```
Console.WriteLine("| 2. Add Cargo ship |");
```

```
Console.WriteLine("| 3. Change properties in your ship |");
```

```
Console.WriteLine("| 4. Remove ship |");
```

```
Console.WriteLine("| 5. 10 show information about the ship 10 |");
```

```
10 Console.WriteLine("| 6. 10 show information about the crew 31 of the ship |");
```

```
Console.WriteLine("| 7. Remove member from ship |");
```

```
Console.WriteLine("| 8. Quit |");
```

```
Console.WriteLine("+-----+");
```

```
Console.Write(" Print number of what 4 you want to do: ");
```

```
k = Console.ReadLine();
```

```
Console.Clear();
```

```
if(k == "8")
```

```
{
```

```

break;

}

switch (k)

{

case "1":

{

PassengerShip pass = new PassengerShip();

Console.WriteLine("\n[Adding Passenger Ship]");

Console.WriteLine(" Print with separating by ',' characteristics of your ship:");

Console.WriteLine(" name 6 of the ship, name of the port, engine power, displacement,");

```

Лістинг Б.6 – Клас Program, аркуш 2

```

Console.WriteLine(" number of passengers, number of sits and capacity of sits");

Console.Write(" -> ");

string character = Console.ReadLine();

pass = pass.AddShips(character) as PassengerShip;

Console.Clear();

Console.WriteLine($"Your ship with individual index: {pass.ShipIndex} was succesfully added
to the {pass.NameOfThePort}");

break;

}

case "2":

{

CargoShip pass1 = new CargoShip();

Console.WriteLine("\n[Adding Cargo Ship]");

```

```

Console.WriteLine(" Print with separating by ',' characteristics of your ship:");

Console.WriteLine(" name 6 of the ship, name of the port, engine power, displacement,");

Console.WriteLine(" load capacity and current load");

Console.Write(" -> ");

string character = Console.ReadLine();

pass1 =pass1.AddShips(character) as CargoShip;

Console.Clear();

Console.WriteLine($"Your ship with individual index: {pass1.ShipIndex} was succesfully
added to the {pass1.NameOfThePort}");

break;

}

case "3":

{ session.ChooseShipYouWantToCange();

Console.Clear();

break;

}

case "4":

{

session.InterfaceforRemovingSips();

break;

Лістинг Б.6 – Клас Program, аркуш 3

}

case "5":

{

```

```
Ship pass2 = new PassengerShip();

Console.Clear();

session.InterfaceOfShowingOfTheShip();

break;

}

case "6":

{

Console.Clear();

session.InterfaceForShowingMember();

break;

}

case "7":

{

session.InterfaceForRemovingMembers();

break;

}

default:

{

Console.Clear();

Console.WriteLine("\n[!] You have entered an incorrect number, try again.");

break;

}

}

}
```



```
}  
  
catch (Exception ex)  
{  
    Console.WriteLine($"\\n[CRITICAL ERROR]: {ex.Message} ");  
}  
  
}  
  
}  
  
}
```

Лістинг Б.6 – Клас Program, аркуш 4

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	stackoverflow.com	0.5%
2	crust.ust.edu.ua	0.3%
3	dspace.onu.edu.ua	0.3%
4	files.eric.ed.gov	0.2%
5	ela.kpi.ua	0.2%
6	itfseafarers.org	0.2%
7	chrislabash-communicationhabitsblog.blogspot.com	0.2%
8	docs.microsoft.com	0.2%
9	onu.edu.ua	0.2%
10	robeseafe.com	0.2%
11	startupbizhub.com	0.2%
12	iq.vntu.edu.ua	0.1%
13	dspace.onu.edu.ua	0.1%
14	docs.com	0.1%
15	docs.microsoft.com	0.1%
16	StudFiles.ru	0.1%
17	travelbag.co.uk	0.1%
18	support.office.com	0.1%
19	uwspace.uwaterloo.ca	0.1%
20	repository.knuba.edu.ua	0.1%
21	wunu.edu.ua	0.1%
22	ru.essays.club	0.1%
23	archive.org	0.1%

#	Джерело	%
24	zavantag.com	0.1%
25	ot.vntu.edu.ua	0.1%
26	pro-prof.com	0.1%
27	digitalcommons.unf.edu	0.1%
28	git.teknik.io	0.1%
29	imdb.com	0.1%
30	dspace.uzhnu.edu.ua	0.1%
31	bl.ocks.org	0.0%



Дякуємо, що перевірили
свій документ за допомогою
Plag!